

CH2 Instructions: Language of the Computer

MIPS Language

```
.text          # Define the program instructions.
main:          # Label to define the main program.
    li $v0,4    # Load 4 into $v0 to indicate a print string.
    la $a0, greeting # Load the address of the greeting into $a0.
    syscall     # Print greeting. The print is indicated by
                # $v0 having a value of 4, and the string to
                # print is stored at the address in $a0.

    li $v0, 10  # Load a 10 (halt) into $v0.
    syscall     # The program ends.

.data          # Define the program data.
greeting: .asciiiz "Hello World" #The string to print.
```

Segment: (程式可以有很多 Seg)

.text: 接受程式碼文字與 label

.data: 接受程式資料與 label

.asciiiz: 存放 ASCII 文字

Instruction Immediate value Label

Commands

li \$s2, 5	Load immediate (constant) value 5 to \$s2
la \$a0, greeting	Load the address at the label into the register
lw \$s1, 12(\$t0)	Load Word (A Byte) from 12(\$t0) to \$s1
↳ A memory address calculate from the base \$t0 + 12 (like: A[12])	
sw \$s3, 12(\$t4)	Store word, 12(\$t4) = \$s3
syscall	Request a system services
add \$s3, \$s1, \$s2	Perform adding \$s3 = \$s1 + \$s2
sub \$s0, \$s0, \$t0	Perform subtraction \$s0 = \$s3 - \$t0
sll \$t0, \$t0, 2	Perform an left shift $t0 = t4 \times 2^n$
move \$s1, \$v0	Copy value in \$v0 into \$s1 (偽指令)
lui \$at, 0xABAB	Load up immediate, (載入上 16bit immediate value)
ori \$at, \$at, 0xCDCD	Or-Immediate val: \$at 與 0xCDCD 做 bit-wise OR

Examples

<pre>.data weight: .asciiiz "weight: " height: .asciiiz "height: " BMI: .asciiiz "BMI: "</pre>	<pre>.text main: li \$v0,4 la \$a0, weight syscall # Print value in *(\$a0) (位址暫存器) li \$v0, 5 syscall # Input value stored in \$v0 li \$v0, 10 syscall # Terminate the program</pre>
--	---

Interrupts

Store interrupt id in **\$v0** (中斷暫存器) and exec it using **syscall** instruction

id	1	4	5	10
Function	Print Int	Print Str	Input	Halt

2.1 Registers

0 *zero* constant 0
1 *at* reserved for assembler
2 *v0* expression evaluation &
3 *v1* function results
4 *a0* arguments
5 *a1*
6 *a2*
7 *a3*
8 *t0* temporary: caller saves
... (callee can clobber 破壞)
15 *t7*
16 *s0* callee saves
... (caller can clobber)
23 *s7*
24 *t8* temporary (cont'd)
25 *t9*
26 *k0* reserved for OS kernel
27 *k1*
28 *gp* pointer to global area
29 *sp* stack pointer
30 *fp* frame pointer
31 *ra* return address (HW)

Temporary registers that are not preserved by the callee (called procedure) on a procedure call

Saved registers that must be preserved on a procedure call (if used, the callee saves and restores them)

CPU 除了以上 32 個 registers 外還有：

No.33: PC (Program Counter)

No.34: EPC (Exceptoin Process Counter)

Computers built on 2 key principles

1. Instructions are represented as numbers
2. Entire programs can be stored in memory to be read or written like numbers (data)

Everything is addressed

Instructions, data, branches and jumps have memory address

A register can hold signed int, unsigned int, pointer (memory address), etc.

Instruction categories

- ⚡ Computational
- ⚡ Load/Store
- ⚡ Jump and Branch
- ⚡ Floating Point
- ⚡ Memory Management
- ⚡ Special

Instructions formats:

R – Format :

OP (5bit)	\$rs (bit)	\$rt (bit)	\$rd (bit)	sa	funct
-----------	------------	------------	------------	----	-------

Example: add, \$rd, \$rs, \$rt

I – Format :

OP (5bit)	\$rs (5bit)	\$rt (5bit)	Immediate (17bit)
-----------	-------------	-------------	-------------------

Example: li \$rs <immediate/constant>(Will be convert to I-Instruction by assembler)

li \$t0, 1234 # Pseudo-instruction
addi \$t0, \$zero, 1234 # Equivalent to

J – Format :

OP (5bit)	Jump Target (27bit)
-----------	---------------------

Example: ? ?

\$rs (R-Source), \$rt (R-Target), \$rd (R-Destination)

2.2 Operations of the Computer Hardware

`add $s0, $s1, $s2` Operator Operand

2.3 Operands of the Computer Hardware

A 32-bit data called a "word"

Arithmetic instructions use register operand

MIPS has 32x 32-bit register (0~31)

Design Principles:

- Regularity makes implementation simpler
- Simplicity enables higher performance at lower cost
- Smaller is faster

Data transfer instructions (Transfer data between mem and reg) e.g. lw, sw

Memory is byte addressed (以byte尋址), each address identifies an 8-bit byte

- Q 2.4 For the MIPS assembly instructions below, what is the corresponding C statement? Assume that the variables f, g, h, i, and j are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays A and B are in registers \$s6 and \$s7, respectively.

```

sll $t0, $s0, 2
add $t0, $s6, $t0
sll $t1, $s1, 2
add $t1, $s7, $t1
lw $s0, 0($t0)
addi $t2, $t0, 4
lw $t0, 0($t2)
add $t0, $t0, $s0
sw $t0, 0($t1)

```

```

# $t0 = f * 4
# $t0 = &A[f]
# $t1 = g * 4
# $t1 = &B[g]
# f = A[f]
$t2 = &A[f+1]
$t0 = A[f+1]
$t0 = A[f+1] + f
$t1 = A[f+1] + f

```

```

int f, g, h, i, j;
int A[100];
int B[100];
f = A[f]
B[g] = A[f+1] + f

```

f	g	h	i	j	A	B
\$s0	\$s1	\$s2	\$s3	\$s4	\$s6	\$s7

```

sll $t0, $s0, 2
add $t0, $s6, $t0
sll $t1, $s1, 2
add $t1, $s7, $t1
lw $s0, 0($t0)
addi $t2, $t0, 4
lw $t0, 0($t2)
add $t0, $t0, $s0
sw $t0, 0($t1)

```

```

$t0 = f * 4
$t0 = &A[f]
$t1 = g * 4
$t1 = &B[g]
f = A[f] ✓
$t2 = &A[f+1]
$t0 = A[f+1]
$t0 = A[f+1] + f
B[g] = A[f+1] + f

```

第二次小考：組合語言翻譯機器語言

R-Format OP (6bit) \$rs (5bit) \$rt (5bit) \$rd (5bit) shamt (5bit) function (6bit)

add rd, rs, rt

special	\$s1	\$s2	\$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$00000010001100100100000000100000_2 = 02324020_{16}$

I-Format OP (6bit) \$rs (5bit) \$rt (5bit) Immediate (16bit)

addi rt, rs, immediate

□ Example

addi \$21, \$22, -50

【addi rt, rs, immediate】

opcode = 8 (look up in table)

rs = \$22 = 22 (register containing operand)

rt = \$21 = 21 (target register)

immediate = -50 (by default, it's decimal in assembly language)

decimal representation

8	22	21	-50
---	----	----	-----

binary representation

001000	10110	10101	111111111001110
--------	-------	-------	-----------------

□ Example

lw \$t0, 1200(\$t1)

【lw rt, immediate(rs)】

opcode = 35 (look up in table)

rs = \$t1 = 9 (base register)

rt = \$t0 = 8 (destination register)

immediate = 1200 (offset)

decimal representation

35	9	8	1200
----	---	---	------

binary representation

100011	01001	01000	0000010010110000
--------	-------	-------	------------------

Immediate value 只可以是 10 base 或 16 base

Immediate value 是 32 bit, 但是 immediate value 只有 16bit, 所以拆成 兩個 register 存

❑ What if immediate is too big to fit in immediate field?

○ Load **Upper** Immediate:

lui register, immediate

○ puts 16-bit immediate in upper half (high order half) of the specified register, and sets lower half to 0s

❑ How to do the following instruction

```
addi $t0, $t0, 0xABABCDCD
内部轉換code
lui $at, 0xABAB
ori $at, $at, 0xCDCD
add $t0, $t0, $at
```

0xABABCDCD	A	B	A	B	C	D	C	D
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1

lui \$at, 0xABAB	A	B	A	B	0	0	0	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0
	1	0	1	0	1	0	1	0

ori \$at, \$at, 0xCDCD	A	B	A	B	C	D	C	D
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1
	1	0	1	0	1	0	1	1

將以上程式翻譯成 Machine code

6bits	5bits	5bits	16bits
lui	\$0 (rs)	\$at (rt)	0xABAB
0011 11	00000	00001	ABAB (16)
3	C	0	ABAB → 0x3C01ABAB

6bits	5bits	5bits	16bits
ori	\$at	\$at	immediate (0xCDCD)
0011 01	00001	00001	CDCD (16)
3	4	2	1
			CDCD → 0x3421CDCD

6bits	5bits	5bits	5bits	5bits	6bits
add	\$t0 (rs)	\$at (rt)	\$t0 (rd)	0	32
0000 00	01000	00001	01000	00000	100000
0	1	0	1	4	0
					2
					0
					→ 0x01014020

Instruction	Example	Meaning	Comment
and	and \$1,\$2,\$3	\$1 = \$2 & \$3	3 reg. operands; Logical AND
or	or \$1,\$2,\$3	\$1 = \$2 \$3	3 reg. operands; Logical OR
nor	nor \$1,\$2,\$3	\$1 = ~(\$2 \$3)	3 reg. operands; Logical NOR
and immediate	andi \$1,\$2,10	\$1 = \$2 & 10	Logical AND with immediate
or immediate	ori \$1,\$2,10	\$1 = \$2 10	Logical OR with immediate
shift left logical	sll \$1,\$2,10	\$1 = \$2 << 10	Shift left by constant
shift right logical	srl \$1,\$2,10	\$1 = \$2 >> 10	Shift right by constant
shift right arithm.	sra \$1,\$2,10	\$1 = \$2 >> 10	Shift right (sign extend)

2.7 Instructions for Making Decisions

Conditional Operations

beq \$rs, \$rt, Label slt \$rd, \$rs, \$rt If true equal to 1

bne \$rs, \$rt, Label slti \$rt, \$rs, constant

I-Format

opcode	rs	rt	immediate
--------	----	----	-----------

Signed vs. Unsigned

Signed comparison: slt, slti

Unsigned comparison: sltu, sltiu

2.8 Supporting Procedures in Computer Hardware

`jr $ra`: 要在跳轉前把 caller 的記憶體位置存到 `$ra`, 以便 callee 返回繼續執行

Callee's Rights, Caller's Responsibilities

Rights	Responsibility
Callees: Use v/a/t registers freely	Save and restore \$s Registers
Caller: Use \$s registers without fear	Saves \$a, \$t registers

/*! Save registers value in \$sp

If a function calles another function, then it's both caller and callee.

Then it need to do all of the rights and responsibilities

Memory Layout

`.text`: Program code, jump (`j` and `jal`) targets

`.data`: Global variables

MIPS 迴圈作法

```
while (save[i] == k) {  
    i += 1;  
}
```

```
Loop: sll $t1, $s3, 2    #offset int is 4 bytes, i*4  
      add $t1, $t1, $s6  # t1=t1+s6, t1=save[i]的i  
      lw  $t0, 0($t1)    # t0 = 0($t1) = save[i]  
      bne $t0, $s5, Exit # t0 == s5 = k  
      addi $s3, $s3, 1   # i += 1  
      j   Loop  
Exit: ...
```

2.10 MIPS Addressing for 32-bit and Addresses

Branch is I-Format

OP (6bit)	\$rs (5bit)	\$rt (5bit)	Immediate (16bit)
-----------	-------------	-------------	-------------------

- 使用相對定址 且指令為 4byte 對齊，所以後兩位一定為 0

→ 可以定址 $16 \times 2 = 18$ bit 的位置 (往前 2^{17} , 往後 2^{17})

Jump Target = $(PC+4) + 25 \times 4$

p.s. 如果跳轉範圍真的太長，組譯器會自動分段跳轉

Jump is J-Format

OP (6bit)	address (26bit)
-----------	-----------------

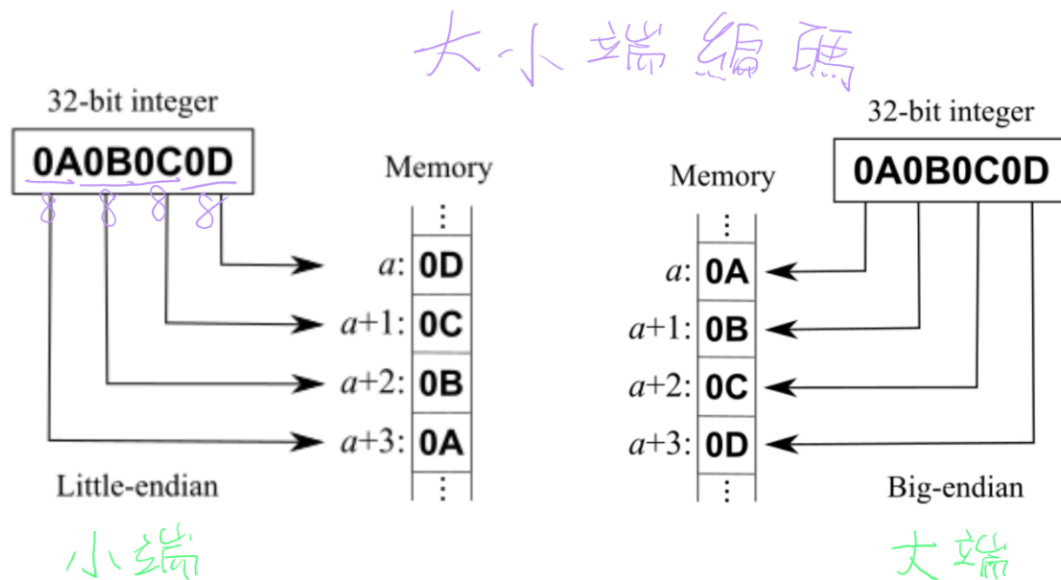
- 只有 26bit 無法紀錄完整 32bit 地址，最多只能紀錄 28 bit (後兩位為0)

→ Pseudodirect addressing: 只覆蓋 PC 的 bit0 ~ bit27

Example:

Loop: sll \$t1, \$s3, 2	80000	j 20000 (80000/4 因為後兩位為0)
add \$t1, \$t1, \$s6	80004	
lw \$t0, 0(\$t1)	80008	
bne \$t0, \$s5, Exit	80012	
addi \$s3, \$s3, 1	80016	
j Loop	80020	
Exit: ...	80024	

Memory Operands



15

Immediate Operands

運算元 (+ - * /)

Immediate: numerical (small) constants

- Often appear (50%) in code
 - put "constants" in memory and load them (沒效率)
- special instructions: Add Immediate

$f = g + 10$ (in C)
addi \$s0,\$s1,10 (in MIPS)
 where \$s0,\$s1 are associated with f, g

MIPS register 0 (\$0 or \$zero) is the constant 0

- hard-wired register, and cannot be overwritten
- Useful for common operations
- addi \$zero, \$zero, 5** will **not** do anything

add \$t2, \$s1, \$zero = move
beq \$r1, \$0, Label

Design Principle 3: Make the common case fast

- Small constants are common
- Immediate operand avoids a load/move instruction

22

2.11 Parallelism and Instructions: Synchronization

如果多個

Exercise

- Q 2.4 For the MIPS assembly instructions below, what is the corresponding C statement? Assume that the variables f, g, h, i, and j are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays A and B are in registers \$s6 and \$s7, respectively.

```

sll $t0, $s0, 2      # $t0 = f * 4
add $t0, $s6, $t0     # $t0 = &A[f]
sll $t1, $s1, 2      # $t1 = g * 4
add $t1, $s7, $t1     # $t1 = &B[g]
lw $s0, 0($t0)        # f = A[f]
addi $t2, $t0, 4      $t2 = &A[f+1]
lw $t0, 0($t2)        $t0 = A[f+1]
add $t0, $t0, $s0     $t0 = A[f+1] + f
sw $t0, 0($t1)        $t1 = A[f+1] + f
    
```

```

int f, g, h, i, j;
int A[100];
int B[100];
    
```

$f = A[f]$

$B[g] = A[f+1] + f$

24

MIPS Data Transfer Instructions

- MIPS arithmetic instructions can read 2 registers, operate on them, and write 1 per instruction
- MIPS data transferring instructions only read or write 1 operand per instruction, and no operation on the operands

Instruction	Comment
sw \$t3, 500(\$t4)	Store word
sh \$t3, 502(\$t2)	Store half 半個 word
sb \$t2, 41(\$t3)	Store byte
lw \$t1, 30(\$t2)	Load word
lh \$t1, 40(\$t3)	Load halfword
lhu \$t1, 40(\$t3)	Load halfword unsigned
lb \$t1, 40(\$t3)	Load byte
lbu \$t1, 40(\$t3)	Load byte unsigned
lui \$t1, 40	Load Upper Immediate (16 bits shifted left by 16)

23